In computer graphics, a **sprite** is a two-dimensional bitmap that is integrated into a larger scene, most often in a 2D video game. Originally, the term *sprite* referred to fixed-sized objects composited together, by hardware, with a background.1(https://en.wikipedia.org/wiki/Sprite_\(computer_graphics\)#cite_note-dadgum-1) Use of the term has since become more general.

Hardware composition of sprites occurs as each scan line is prepared for the video output device, such as a cathode-ray tube, without involvement of the main CPU and without the need for a full-screen frame buffer.1(https://en.wikipedia.org/wiki/Sprite_\(computer_graphics\)#cite_note-dadgum-1) Sprites can be positioned or altered by setting attributes used during the hardware composition process. The number of sprites which can be displayed per scan line is often lower than the total number of sprites a system supports. For example, the Texas Instruments TMS9918 chip supports 32 sprites, but only four can appear on the same scan line.

The CPUs in modern computers, video game consoles, and mobile devices are fast enough that bitmaps can be drawn into a frame buffer without special hardware assistance. Beyond that, GPUs can render vast numbers of scaled, rotated, anti-aliased, partially translucent, very high resolution images in parallel with the CPU.

2.5D (basic pronunciation **two-and-a-half dimensional, two-point-five-d**) perspective refers to gameplay or movement in a video game or virtual reality environment that is restricted to a two-dimensional (2D) plane with little to no access to a third dimension in a space that otherwise *appears* to be three-dimensional and is often simulated and rendered in a 3D digital environment.

This is related to but separate from pseudo-3D perspective (sometimes called three-quarter view when the environment is portrayed from an angled top-down perspective), which refers to 2D graphical projections and similar techniques used to cause images or scenes to simulate the appearance of being three-dimensional (3D) when in fact they are not.

By contrast, games, spaces or perspectives that are simulated and rendered in 3D and used in 3D level design are said to be *true 3D,* and 2D rendered games made to appear as 2D without approximating a 3D image are said to be *true 2D*.

Common in video games, 2.5D projections have also been useful in geographic visualization (GVIS) to help understand visual-cognitive spatial representations or 3D visualization.1(https://en.wikipedia.org/wiki/2.5D#cite_note-MacEachren-1)

The terms *three-quarter perspective* and *three-quarter view* trace their origins to the three-quarter profile in portraiture and facial recognition, which depicts a person's face that is partway between a frontal view and a side view.2(https://en.wikipedia.org/wiki/2.5D#cite_note-2)

Computer graphics

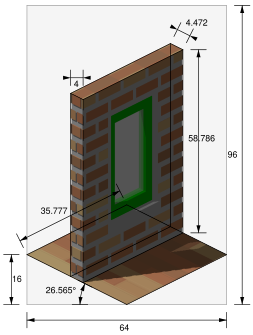
Axonometric and oblique projection

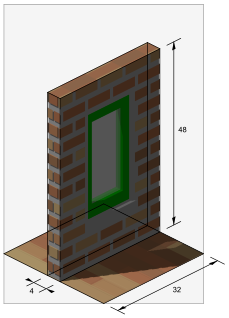
See also: Isometric computer graphics

Lincity tiles 2D axonometric graphical elements to form a pseudo-3D game environment.

In axonometric projection and oblique projection, two forms of parallel projection, the viewpoint is rotated slightly to reveal other facets of the environment than what are visible in a top-down perspective or side view, thereby producing a three-dimensional effect. An object is "considered to be in an inclined position resulting in foreshortening of all three axes",3(https://en.wikipedia.org/wiki/2.5D#cite_note-WPCleanerAuto1-3) and the image is a "representation on a single plane (as a drawing surface) of a three-dimensional object placed at an angle to the plane of projection."3(https://en.wikipedia.org/wiki/2.5D#cite_note-WPCleanerAuto1-3) Lines perpendicular to the plane become points, lines parallel to the plane have true length, and lines inclined to the plane are foreshortened.

They are popular camera perspectives among 2D video games, most commonly those released for 16-bit or earlier and handheld consoles, as well as in later strategy and role-playing video games. The advantage of these perspectives is that they combine the visibility and mobility of a top-down game with the character recognizability of a side-scrolling game. Thus the player can be presented an overview of the game world in the ability to see it from above, more or less, and with additional details in artwork made possible by using an angle: Instead of showing a humanoid in top-down perspective, as a head and shoulders seen from above, the entire body can be drawn when using a slanted angle; turning a character around would reveal how it looks from the sides, the front and the back, while the top-down perspective will display the same head and shoulders regardless.





Anatomy of an axonometric sprite. 2D sprite coordinates are on the left. 3D model coordinates are on the right.

There are three main divisions of axonometric projection: *isometric* (equal measure), *dimetric* (symmetrical and unsymmetrical), and *trimetric* (single-view or only two sides). The most common of these drawing types in engineering drawing is isometric projection. This projection is tilted so that all three axes create equal angles at intervals of 120 degrees. The result is that all three axes are equally foreshortened. In video games, a form of dimetric projection with a 2:1 pixel ratio is more common due to the problems of anti-aliasing and square pixels found on most computer monitors.

In oblique projection typically all three axes are shown without foreshortening. All lines parallel to the axes are drawn to scale, and diagonals and curved lines are distorted. One tell-tale sign of oblique projection is that the face pointed toward the camera retains its right angles with respect to the image plane.[*clarification needed*]

Benefits

In an application where many small textures are used frequently, it is often more efficient to store the textures in a texture atlas which is treated as a single unit by the graphics hardware. This reduces both the disk I/O overhead and the overhead of a context switch by increasing memory locality. Careful alignment may be needed to avoid bleeding between sub textures when used with mipmapping and texture compression.

In web development, images are packed into a sprite sheet to reduce the number of image resources that need to be fetched in order to display a page.2(https://en.wikipedia.org/wiki/Texture_atlas#cite_note-2)

In computer graphics, a **mipmap** (*mip* being an acronym of the Latin phrase *multum in parvo*, meaning "much in little"1(https://en.wikipedia.org/wiki/Mipmap#cite_note-staff-1)) is a pre-calculated, optimized sequence of images, each of which has an image resolution which is a factor of two smaller than the previous.2(https://en.wikipedia.org/wiki/Mipmap#cite_note-2)[[3]](https://en.wikipedia.org/wiki/Mipmap#cite_note-3)[[4]](https://en.wikipedia.org/wiki/Mipmap#cite_note-4) Their use is known as *mipmapping*.

They are intended to increase rendering speed and reduce aliasing artifacts. A high-resolution mipmap image is used for high-density samples, such as for objects close to the camera; lower-resolution images are used as the object appears farther away. This is a more efficient way of downscaling a texture than sampling all texels in the original texture that would contribute to a screen pixel; it is faster to take a constant number of samples from the appropriately downfiltered textures. Since mipmaps, by definition, are pre-allocated, additional storage space is required to take advantage of them. They are also related to wavelet compression.

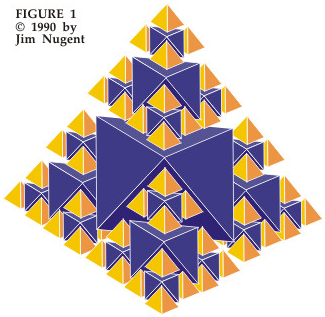
Mipmaps are widely used in 3D computer games, flight simulators, other 3D imaging systems for texture filtering, and 2D and 3D GIS software. Mipmap textures are used in 3D scenes to decrease the time required to render a scene. They also improve image quality by reducing aliasing and Moiré patterns that occur at large viewing distances,5(https://en.wikipedia.org/wiki/Mipmap#cite_note-5) at the cost of 33% more memory per texture.

The origin of the term mipmap is an initialism of the Latin phrase *multum in parvo* ("much in little"), and map, modeled on bitmap.1(https://en.wikipedia.org/wiki/Mipmap#cite_note-staff-1) The term *pyramids* is still commonly used in a GIS context. In GIS software, pyramids are primarily used for speeding up rendering times.

# **Pascal's Pyramid O**r Pascal's Tetrahedron?	
By Jim Nugent 23111 Central Park Avenue Richton Park, Illinois 60471-2601 lastnamefirstinitial at ameritech.net This is a copyrighted document. The text, charts, graphs and illustrations are covered by various copyrights ©1990, 1991, 1996, 1997, 1999, 2006, & 2007	
## ABSTRACT A lattice of octahedra and tetrahedra (oct-tet lattice) is a useful paradigm for understanding the structure of Pascal's pyramid, the 3-D analog of Pascal's triangle. Notation for levels and coordinates of elements, a standard algorithm for generating the values of various elements, and a ratio method that is not dependent on the calculation of previous levels are discussed. Figures show a bell curve in 3 dimensions, the association of elements to primes and twin primes, and the values of elements mod(x) through patterns arranged in triangular plots. It is conjectured that the largest factor of any element is less than the level index. This 22K article contains illustrations and animations that are quite large. These files can take a few minutes to download if you access the Internet with a modem or slow connection.	

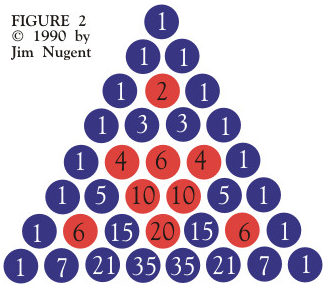
Pascal's Pyramid Or Pascal's Tetrahedron?

I. Introduction



In a 1966 Scientific American column, Martin Gardner mentions in passing that there are, ". . . endless variants on the [Pascal's] triangle, and many ways to generalize it, such as building it in tetrahedral form to give the coefficients of trinomial expansions."

One of the illustration of Pascal's triangle, included with that article shows the first 100 rows of the triangle with odd and even numbers represented as dots. The pattern of dots representing odd and even numbers in the two dimensional Pascal's triangle correspond to the octahedral and tetrahedral sections of an oct-tet lattice (see figures 1 and 2). The octahedron faces of the lattice map to the even dots while the tetrahedron faces map to the odd dots.

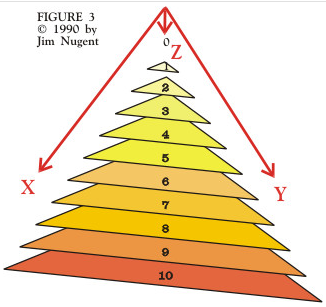


If we construct a three dimensional Pascal's triangle will the distribution of odd and even values for the elements in the interior correspond to the octahedral and tetrahedral components of an oct-tet lattice?

A Galton board simulation, rolling marbles through a triangular maze, generates values that approximate the values of coefficients of a binomial expansion. At each fork in the maze the marble has a 50-50 chance to go either way until it falls into bins at the bottom of the maze in proportions that roughly approximate a bell-shaped curve. The Galton Board can be simulated with a computer program that is a variation of the one used to generate the 3-D analog of Pascal's triangle.

II. Precursors

Stephen Mueller published a paper in 1969 on **Recursions Associated With Pascal's Pyramid**. In Mueller's definition, "Pascal's pyramid is the three-faced pyramidal array of coefficients in the expansion of the trinomial, (a+b+c)j, such that the coefficients of (a+b+c)j are systematically placed beneath those of (a+b+c)j-1, resulting in a Pascal triangle on each of the three faces." Mueller labels the initial plane, containing one term, as level 0 (see figure 3) which turns out to be a good choice of notation. Terms on higher levels are expressed as the product of two binomial coefficients.



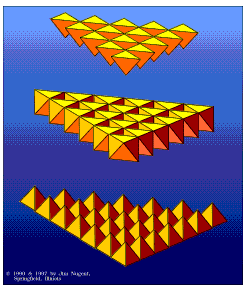
John Staib and his son Larry published **The Pascal Pyramid** in 1978. Like Mueller, the Staibs also used a stack of successively larger triangles to form a pyramid (seefigure 3). They developed a coordinate notation for each point on each triangular level, a system for determining which of the small triangles were centered above a point on the next level, that the sums of the trinomial coefficients associated with the vertices of an upper triangle were equal to the trinomial coefficient found directly below that triangle, that a trinomial expansion could be generalized from a binomial expansion and that a BASIC computer program could be written to generate the coefficients of (x+y+z)n in the form of layers of a pyramid. In a 1938 book, Steinhaus gives a short example of a method of determining winners in a three way election with a similar notation except that he breaks the triangle into hexagonal cells.

In 1986, Bollinger attempted to lay pyramid power to rest with, "It would seem that in spite of the appeal of an array for multinomial coefficients similar to the triangle for binomials, one is better off for most purposes using a convenient algorithm to generate the m-part compositions of n, from which the exponents on the xi and the multinomial associated with a given term are immediately available."

III. The Proper Paradigm

Two key concepts needed for a clearer understanding of the 3-D analog of Pascal's triangle are missing from these earlier works.

Figure 4

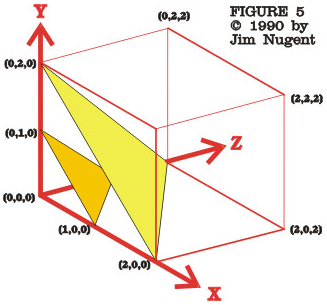


First, the octahedron-tetrahedron lattice is a more accurate 3-D analog for the 2-D Pascal's triangle. A triangular base pyramid is descriptive of outward shape but lacks any ability to illuminate the interconnectedness of multiple subdivisions. The levels of the 3-D analog of Pascal's triangle are connected by an octahedron-tetrahedron lattice. Some of the tetrahedra are oriented with a point up and some with a face up (see figure 4). Loeb states it clearly:

(Link to standalone Octahedron-Tetrahedron Animation (figure 4))

"It is not possible to fill space with regular tetrahedra [or pyramids] only; four tetrahedra and an octahedron are necessary to produce a tetrahedron having eight times the volume (twice the linear dimension) of the original tetrahedron."

In order to fill space octahedra are used in combination with tetrahedra (see figure 1). Subdividing the pyramid into these two components makes it a simple task to model the 3-D analog of Pascal's triangle.



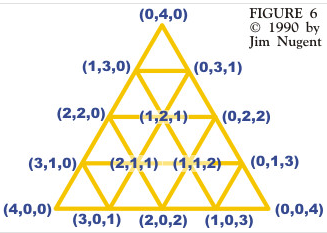
Secondly, Pascal's pyramid (or tetrahedron) can be modeled with ordinary Cartesian coordinates. A tetrahedron is point-for-point congruent to a cube-corner (see figure 5). If you visualize the X, Y, and Z axes as the three legs of a collapsible photo tripod, the transformation from cube corner to oct-tet lattice is as like changing the angle between legs from 60 to 90 degrees. As you raise the tripod the legs get longer but the angles between them grow more acute.

IV. The Octahedron-Tetrahedron Lattice

Like Bollinger [6], Staib and Staib [4] state that, ". . .the three-dimensional nature of the Pascal pyramid makes it awkward to use for hand calculating the trinomial coefficients." Staib and Staib [4] and Mueller [3] both use illustrations of pyramids containing planes and levels with no interconnections between levels. Drawing a tetrahedron, let alone a subdivided tetrahedral lattice can be hard. Without actual physical models it can be daunting to visualize how tetrahedra in combination with octahedra might provide the needed paradigm for interconnecting the pyramid's levels (see figure 3 and 4). Oct-tet lattices are also harder to picture in two-dimensioned figures and illustrations because most of our drawing conventions are based on viewing cubic forms.

Looking for patterns among 2 or 3 thousand forty digit numbers is not easy work. "Computer graphics is an excellent method by which patterns in Pascal's pyramid can be made obvious to both the mathematician and interested layperson," according to Pickover and Khorasani . The figures in the following sections are computer generated visualizations that highlight certain information while masking other things. To decide if a 50 digit number is odd or even we only need to examine one digit, the last one.

The oct-tet lattice provides not only a mental model but also a physical model of a 3-D analog of Pascal's triangle. A group of small models made of toothpicks, soda straws, or cardboard can prove useful in order to "think" about tetrahedra.



V. Basic Notation

The notation in figure 6 is suitable, it agrees closely with Staib and Staib's notation for coordinates on any level while it uses Mueller's scheme for numbering the levels. The first level is set at zero. This is convenient because the value of the second element in the first row at higher levels is equal to the level and the sum of the coordinates for each point on a level is equal to the level index.

True BASIC Program to calculate Pascal's Tetrahedron Element Values.

VI. Some Interesting Properties

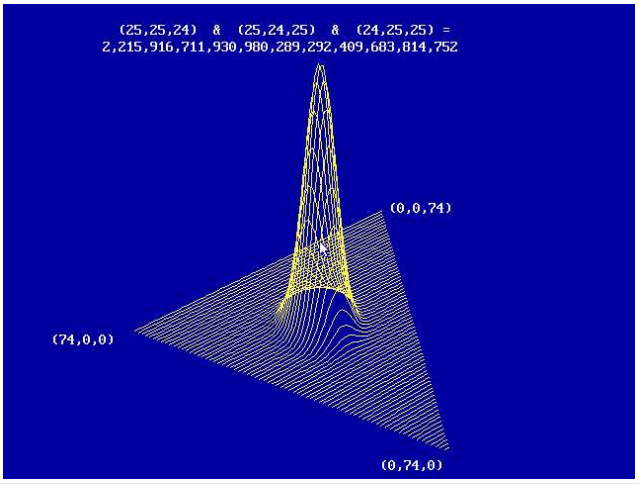
A. A Fully Rounded Bell Curve

Any row in Pascal's triangle yields the coefficients of a binomial expansion. A plot of the coefficients gives us the familiar, flat, two dimensional "bell" curve. This normal distribution can be modeled with a Galton Board simulation or with a series of heads or tails coin tosses, each with a probability of 1/2.

The triangular levels of the 3-D analog yield the coefficients of trinomial expansions. A plot of the coefficients gives us a three-dimensional "bell" curve (see figure 7 and 8). The 3-D version can also be modeled. By constructing the edges of the octahedra and tetrahedra in an oct-tet lattice with hollow tubes it would be possible to construct a working 3-D version of a Galton Board. Here we would have a series of decisions with three possible outcomes, each with a 1/3 probability.

Figure 7.

A three dimensional bell curve. Level 74 of Pascal's Tetrahedron

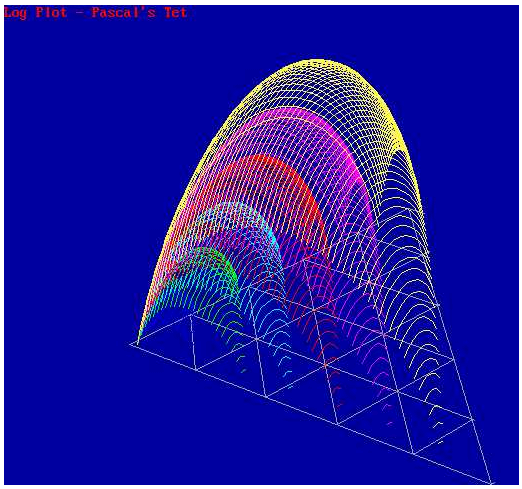


True BASIC Program to generate Bell Curve Plot

True BASIC Program to generate Log Bell Curve Plot

The three edge rows of the 3-D "bell" curve (see figure 7) and the three flat faces of the log plot (see figure 8 which shows log plots of values for 6 different levels) are 2-D "bell" curves. The elements on three of the four faces of the tetrahedral 3-D analog are the same as the elements of the 2-D Pascal's triangle. Pascal's triangle is contained in the 3-D version.

> ### Figure 8.
>
> Log plot bell curve for 6 levels of Pascal's Tetrahedron
>



>

B. Factoring Elements

It appears that each element of a level factors into values that are less than the level number (click for proof). A random example. The value of the central element on level 1152 is a 547 digit number (click to see figure 9). It has 227 factors, the largest being 1151. This 547 digit number has factors that included 85% of the primes below it's level index of 1152 (click to see figure 10).

It took 23 hours to come up with this central element's value and another 11 hours to factor it, on a standard IBM PC. For example, Pomerance states, ". . . factoring is hard. It has been estimated conservatively . . . that to factor certain composite n with 75 decimal digits, using the fastest factoring algorithm known on an imaginary computer faster than any now in existence would take about 15 weeks. For certain numbers with 100 digits it could take a life time." The elements of the 3-D analog of Pascal's triangle are exceptions to Pomerance's statement.

The numbers mentioned above are so large that few personal computer languages can handle them. The True BASIC language along with two language extension packages, the 3-Dimensional Graphics and the Mathematician's Toolkit, were used to compute the examples in this article. The Hugelib library allows for arithmetic of arbitrary precision, up to 100,000 digits by packing numbers into strings.

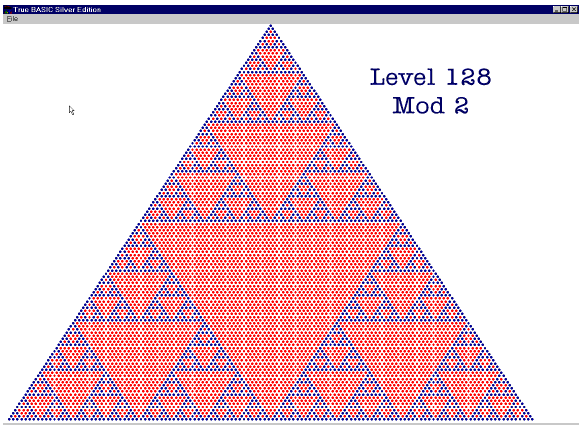
C. Odds and Evens

A number of interesting 2-D (and 3-D) patterns can be generated by representing the values of the elements of a level with colored dots. These are similar to the patterns of odd and even distribution within Pascal's triangle . In the 3-D analog the odds and evens are distributed into octahedral and tetrahedral components. When we plot the distributions of other multiples for various levels we obtain other interesting patterns (see figure 11).

True BASIC Program to generate Mod Plots of Pascal's Tetrahedron Element Values

Figure 11

A screen shot of level 128 - modulus 2 showing odd and even numbers resembles a Sierpinski gasket



The mod(2) patterns for the slices of levels 7, 15, 31, 63, 127 . . . are the same at the patterns for the three "exterior" faces of the tetrahedron (see figure 2). For a tetrahedron that is one level less than a power of two, all four faces have the same pattern.

D. Plus or Minus One Primes

The values of the elements of this 3-D analog of Pascal's triangle factor into many small primes, they are divisible by a high percentage of integers (level 75 element values). These same element values plus or minus one have few divisors and few factors. Upon investigation we find that many of them are prime. Many elements are one more or less than a prime number. Some are between twin primes (see figure 12).

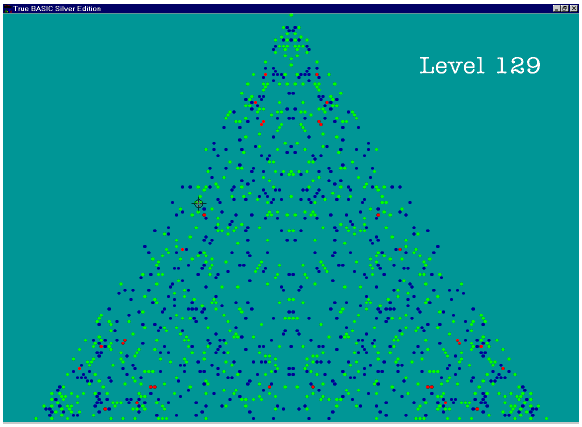
Because computer languages for personal computers can handle about 15 significant digits you can calculate values for elements up to about level 35 in a straight forward manner. Program and array size limitations in many popular languages such as BASIC limit exploration of the 3-D analog of Pascal's triangle to the first 18 levels. Brute force factoring methods also become unbearably time consuming at these levels.

Figure 12 a

To explore above level 18 I switched from using brute force factoring as a prime test to using Fermat's Little Theorem with a base of 2 as a composite test. This

A visualization of level 129 of Pascal's Tetrahedron. Elements are color coded.

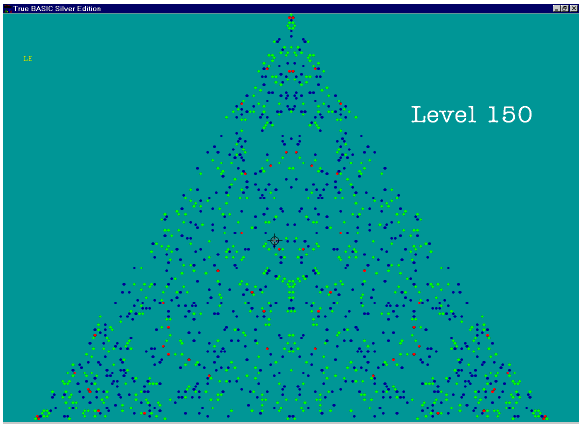
- A red dot is an element that is between twin primes
- A green dot is an element that is one less than a prime number
- A blue dot is an element that is a prime number plus 1



can be stated: if bn - b is a multiple of n, then n is a composite number. Above level 18 the composites I've identified are composite, the primes are another story. Some might be pseudoprimes, some might be Charmichael numbers. It will take more than a personal computer to be able to tell which is which.

There have been some estimates of the number of pseudoprimes. "The scarcity of pseudoprimes to the base 2 among all the numbers smaller than 20 billion suggests that any number that passes the Fermat test to base 2 is likely to be prime," according to Pomerance . Of course a good proportion of the values from the 3-D analog of Pascal's triangle that I've looked at are above Pomerance's 20 billion limit. The central values of level 74 are approximately 3.4x1034.

Figure 12b



True BASIC Program to record images
like those on the right

True BASIC Program to playback images
like those on the right

Exploration above level 18 would benefit from faster algorithms and computer systems that are more powerful. Since any element, from one of the levels of the tetrahedral array, is easy to factor (all factors have been less than the level index for every element factored to date). It might be possible to use Dixon's algorithm which is based on a complete factorization of n-1, or other newer algorithms based on partial factorizations of n-1 or n+1.

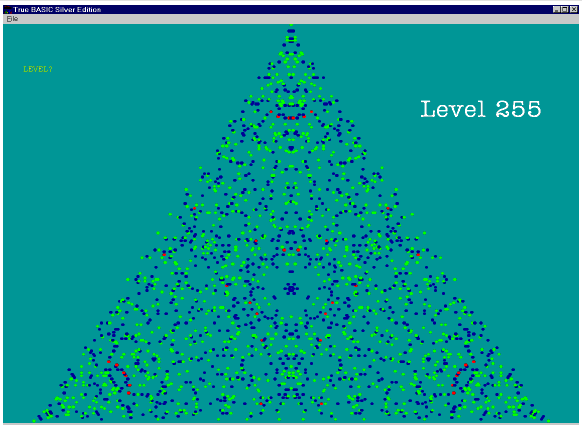


Figure 12c

The six elements of Pascal's Tetrahedron coded in blue at the very center of level 255 are 120 digits numbers. I've broken that number into groups of ten digits for ease of inspection:

1481381295 1722496842 2500487141
4311719641 9437870393 1200589484
8675243369 2295824399 2828691173
4768379843 4796671471 2573495000

If you subtract one from this 120 digit number (change the final 5000 to 4999 you get a prime number). The ASCII text file listing the elements of level 255 is over 300 pages long.

## Figure 12d - Quicktime animation Click below to view a 3,125 Kilobyte Quicktime animation (320 by 240) of primes on levels 1 to 150 similar to the 3-D illustration above. You might need to save this to disk and then open it with Quicktime. I'm just learning this stuff. Go to Apple to get Quicktime Viewer Plugin	

E. The Oct-Tet Algorithm

The first line of code sets point(0,0,0) equal to 1 by definition. Next we assign a value of 1 to any point (on

a **vertex**) that has 2 coordinates of zero. If an element is not the initial point or a vertex point then we check to see if it's on the tetrahedral face. If it is we sum the two values from the level above for any element that is on the **edge** of a level. The final ELSE statement assigns a value for all the points **not on an edge or vertex**, setting them equal to the sum of the 3 elements from the level above.

```
Let PT(0,0,0) = 1
   FOR level = 1 TO top
      FOR x = level TO 0 STEP -1
         FOR y = (level - x) TO 0 STEP -1
            FOR z = (level - (x+y)) TO 0 STEP -1
            IF x = 0 AND y = 0 OR x = 0 AND z = 0
            OR y = 0 AND z = 0 THEN
               LET pt(x,y,z) = 1
            ELSE IF x = 0
               LET pt(x,y,z) = pt(x,y-1,z) + pt(z,y,z-1)
            ELSE IF y = 0 THEN
               LET pt(x,y,z) = pt(x-1,y,z) + pt(z,y,z-1)
            ELSE IF x = 0 THEN
               LET pt(x,y,z) = pt(x-1,y,z) + pt(z,y-1,z)
            ELSE
               LET pt(x,y,z)=pt(x-1,y,z)+pt(x,y-1,z)+pt(x,y,z-1)
            NEXT z
      NEXT y
   NEXT z
NEXT level
```

F. An Alternate Algorithm

In order to explore above level 18 of the 3-D analog of Pascal's triangle on a PC it is necessary to use a language that uses more than 64K of memory and can handle more than 16 digit integers. Since each level is dependent on only one previous level it would also be possible to explore higher levels if we could find an algorithm that made more sparing use of computer memory.

Alternating between two flat arrays rather than creating one large cubic array would be one solution. The drawback is that it would involve lots of coordinate manipulation. On the other hand mapping an octahedron-tetrahedron lattice onto the standard x,y,z cubic coordinate system is wasteful of computer memory because we can only store 5456 elements in the 27,000 storage locations in an array 30 by 30 by 30.

Another method of reaching higher levels in the 3-D analog of Pascal's triangle (also true of Pascal's triangle) is based on the fact that each element of the tetrahedral array is related by a simple whole number ratio to the 12 elements around it (see figure 13). The value of any element can be calculated from any one of it's neighbors. We can start at any vertex element (which has a value of 1, by definition) on any level and apply a ratio calculation, proceeding from element to element, each time incrementing the denominator and decrementing the numerator of the interval ratio by 1. Using this system the need for array storage is cut drastically.

Figure 13

Each element of Pascal's Tet is related to every other element by a simple ratio. Also note the relationships between ratios going across and up and down.

Even with shortcuts, the time needed to calculate the values of elements and run the Fermat test quickly rises (figure 14 is not on WWW). Level 75 took 7 hours to compute on a personal computer with a 10 megahertz 80286 processor. For the first 75 levels the ratio of primes that are equal to the value of an element plus or minus 1 appears to hold steady (figure 15 is not on WWW).|

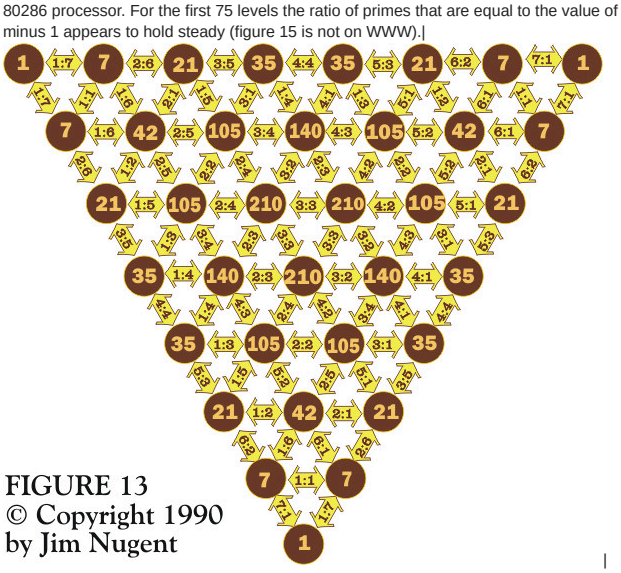


FIGURE 13
© Copyright 1990
by Jim Nugent

|

G. Some Other Properties

The value of any element is equal to the number of different paths from the apex of the tetrahedron to that element. In Pascal's triangle this is the basis of "Taxicab" geometry.

The sum of the unique elements on any level equals 3 to the power of the level index. For instance level 7 has eight unique elements; 1, 7, 21, 35, 42, 105, 140, and 210. These add up to 2187 or 3 to the seventh power.

VII. Summary

The three dimensional analog of Pascal's triangle is of interest for the same reasons that Pascal's triangle is of interest. It touches on number theory, the distribution of primes and twin primes, divisors, factors, combinatorics, and geometry.

The patterns to be found in Pascal's triangle and it's 3-D analog seem endless.

References

Check the complete article in Computers and Graphics: An International Journal of Applications in Computer Graphics, Volume 15, Number 2 1991 for more information. Pergamon Press ISSN 0097-8493

Footnotes

> Gardner, Martin. (1966, December) The multiple charms of Pascal's triangle, Scientific American. pp. 128-132
>
> Dewdney, A. K. (1985, April) Five easy pieces for a do loop and random-number generator [Computer Recreations]. Scientific American. p. 22.
>
> Mueller, Stephen. (1969, Spring). Recursions associated with Pascal's pyramid. Pi Mu Epsilon Journal, (Vol 4, #10). pp. 417-422.
>
> Staib, John & Staib, Larry. (1978, September). The Pascal Pyramid. Mathematics Teacher. pp. 505-560.
>
> Steinhaus, H. (1939). Mathematical snapshots. Leipzig: G. E. Stechert & Co. p. 30.
>
> Bollinger, Richard C. (1986, May) A note on Pascal-t triangles, multinomial coefficients, and Pascal pyramids, The Fibonacci Quarterly. pp. 140-144.
>
> Loeb, Arthur L. (1975) "Contribution to Synergetics," in Synergetics, by R. Buckminster Fuller. MacMillan (New York, 1975). p. 837.
>
> Pickover, C and Khorasani, E. (1990) Infinite Triangular Arrays. Leonardo 23(3). In press.
>
> Pomerance, Carl. (1981) Recent Developments in Primality Testing Math Intelligencer 3. pp. 97-105.
>
> Pomerance, Carl. (1982, June) The Search for Prime Numbers, Scientific American. p. 141.
>
> Dixon, J. D. (1981) Asymptotically fast factorization of Integers. Math. Comp. 36. pp. 255-260.

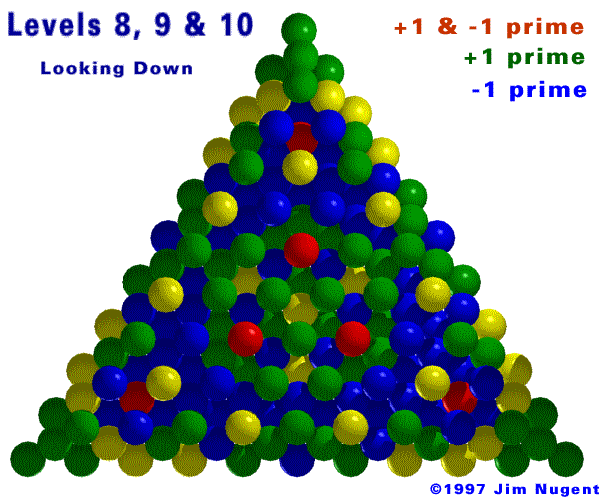
Back to the Top

Visualizing Plus/Minus Primes on Multiple Levels

A Thought Experiment

The elements Pascal's Tet are very divisible, but if we add or subtract one from an element's value we get some very undivisible numbers, i.e., primes or at least something like primes.

Visualize Pascal's Tetrahedron as a stack of marbles. One marble for each element. A triangle of marbles for each level of the tetrahedron. Code the marbles red, green and blue if the plus or minus one value is prime. If not make it yellow. Levels 8, 9 & 10 in the 3-D illustration below exhibit some interesting patterns.



Here are the numerical values that go with the various "marbles" Values for level 8, 9 & 10
Click here to view levels 32, 33, & 34 - this will take a minute since the GIF file is about 100K in size.
Click here to view levels 32, 33, & 34 - this is even higher resolution - about 300K in size.

When working with libGDX (or any other OpenGL based system), you will have to deal with various coordinate systems. This is because OpenGL abstracts away device dependent units, making it more convenient to target multiple devices and to focus on game logic. Sometimes you may need to convert between coordinate systems for which libGDX offers various methods.

It is crucial to understand in which coordinate system you are working. Otherwise it can be easy to get confused and to make assumptions which aren't correct.

On this page the various coordinate systems are listed. It is highly recommended to first get familiar with the concept of Cartesian coordinate systems, which is the most widely used one.

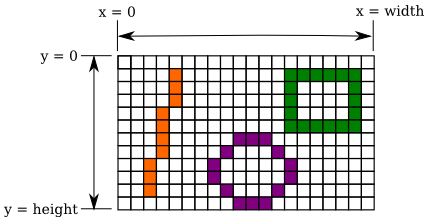
- Touch coordinates
- Screen or image coordinates
- Pixmap and texture coordinates

- Normalized render coordinates
- Normalized texture (UV) coordinates
- World coordinates
- GUI/HUD coordinates
- Game coordinates

Touch coordinatesPermalink

Units	pixels
System	y-down
Type	integer (can't be fractional)
Range	(`0` , `0`) (upper left corner) to (`Gdx.graphics.getWidth()-1` , `Gdx.graphics.getHeight()-1`) (lower right corner)
Usage	touch/mouse coordinates
Dependence	device specific

Starts at the upper left *pixel* of the (application portion of the) physical screen and has the size of the (application portion of the) physical screens width and height in pixels.



Each coordinate is an index in the 2D array of this grid, representing a physical pixel on the screen. Therefore these coordinates are always represented as integers, they can't be fractional.

This coordinate system is based on the classic representation of the display, which is usually also closest to the device/OS specific implementation. If you're familiar with canvas graphics or basic image editors, then you are probably already familiar with these coordinates. You might even lean towards using these as your default/favorite, which you shouldn't.

Whenever working with mouse or touch coordinates, you'll be using this coordinate system. You typically want to convert these coordinates as soon as possible to a more convenient coordinate system. E.g. the `camera.unproject` or `viewport.unproject` method let's you convert them to world coordinates (see below).

Screen or image coordinatesPermalink

Units	pixels
System	y-up
Type	integer (can't be fractional)
Range	(`0` , `0`) (lower left corner) to (`Gdx.graphics.getWidth()-1` , `Gdx.graphics.getHeight()-1`) (upper right corner)
Usage	viewport, scissors and pixmap
Dependence	device/resource/asset specific

This is OpenGL's counterpart to touch coordinates; that is: it is used to specify (index) a pixel of the (portion of the) physical screen. It is also used as indexer for an image in memory. Likewise, these are integers, they can't be fractional.

The only difference between touch and screen coordinates is that touch coordinates are y-down, while screen coordinates are y-up. Converting between them is therefore quite easy:

Copy code `y = Gdx.graphics.getHeight() - 1 - y;`

You typically use these coordinates to specify which portion of the screen to render onto. For example when calling `glViewport` , `glScissor` or manipulating a pixmap (see next). In the majority of use-cases you don't need this coordinate system a lot, if any, and it should be isolated from your game logic and its coordinate system. The `camera.project` and `viewport.project` methods can be used to convert world units to screen coordinates.

Pixmap and texture coordinatesPermalink

Pixmap coordinates are an exception. Pixmaps are commonly used to upload texture data. For example when loading a PNG image file to a texture, it is first decoded (uncompressed) to a Pixmap, which is the raw pixel data of the image, then it is copied to the GPU for use as texture. The texture can then be used to render to the screen. It is also possible to modify or create a pixmap by code, e.g. before uploading as texture data.

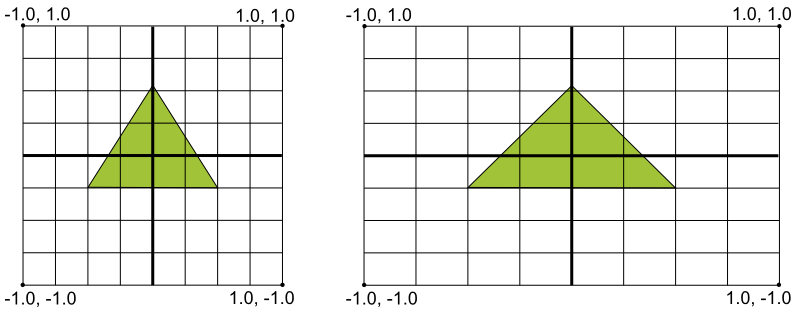
The "problem" with this is that OpenGL expects the texture data to be in image coordinates, which is y-up. However, most image formats store the image data comparable to touch coordinates, which is y-down. libGDX does not translate the image data between the two (which would involve copying the image line by line), instead it simply copies the data as is. This practically causes a Texture loaded from Pixmap to be up-side-down.

To compensate for this up-side-down texture, `SpriteBatch` flips the texture (UV) coordinates (see below) on the y axis when rendering. Likewise, fbx-conv has the option to flip texture coordinates on the y axis as well. However, when you use a texture which isn't loaded from a pixmap, for example a Framebuffer, then this might cause that texture to appear up-side-down.

Normalized render coordinatesPermalink

Units	one
System	y-up
Type	floating point
Range	(`-1` , `-1`) (lower left corner) to (`+1` , `+1`) (upper right corner)
Usage	shaders
Dependence	none

The above coordinate systems have one big issue in common: they are device specific. To solve that, OpenGL allows you to use a device independent coordinate system which is automatically mapped to screen coordinates when rendering. This coordinate system is normalized in the range [-1,-1] and [+1,+1] with (0,0) exactly in the center of the screen or framebuffer (the render target).



The *vertex shader* outputs (`gl_Position`) its coordinates in this coordinate system. But other than that, you should never have to use this coordinate system in a practical use-case. It is sometimes used in tutorials and such, though, to show the basics.

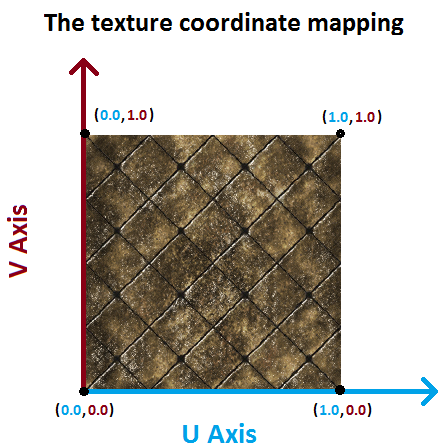
It might be worth to note that the normalization does not respect the aspect ratio. That is: the scale in the X direction does not have to match the scale in the Y direction. They are both within the range of `-1` to `+1`, regardless aspect ratio. It is up to the application to decide how to deal with various aspect ratios (see world units, below).

The coordinates are floating point and no longer indexers. The device (GPU) will map these coordinates to the actual screen pixels using rasterisation. A good article (although targeting DirectX it also applies to OpenGL) for more information on that can be found here.

Normalized texture (UV) coordinatesPermalink

Units	one
System	y-up
Type	floating point
Range	(`0` , `0`) (lower left corner) to (`1` , `1`) (upper right corner)
Usage	shaders, mesh, texture region, sprite
Dependence	none

Likewise to the normalized render coordinates, OpenGL also uses normalized texture coordinates. The only difference is that these ranges from [0,0] to [1,1]. Depending on the specified wrap function, values outside that range will be mapped within that range.



These coordinates are also called **UV coordinates**. In many use cases you don't have to deal with them. Typically these values are stored in the mesh or `TextureRegion` .

The use of normalized texture coordinates is very important, because it makes them independent of the asset size. Or in other words: it allows you to replace your assets with a scaled down or scaled up version, without having to modify the UV coordinates. An example where this is used are mipmaps.

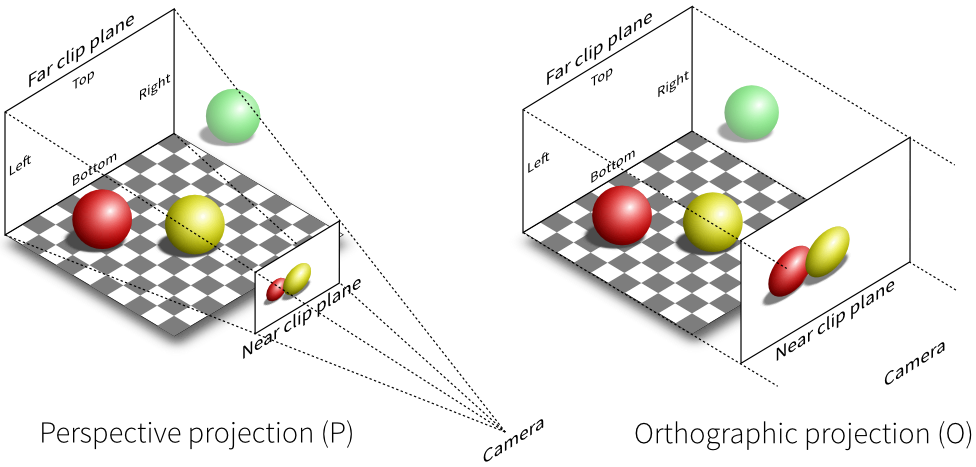
When rendering, the GPU converts the UV coordinates to a texel (texture pixel). This is called "texture sampling" and is based on the texture filtering.

World coordinatesPermalink

Units	application specific, e.g. SI Units
System	application specific, but usually y-up
Type	typically floating point
Range	application specific
Usage	game logic
Dependence	game/application

Typically, your game logic should use a coordinate system which best fits the game logic. It should not depend on device or asset size. For example, a commonly used unit is meters.

The world coordinates are converted, in the vertex shader, to normalized render coordinates. The Camera or Viewport is used to define the strategy on how to do that. For example, to maintain aspect ratio, black bars can be added. The camera is used to calculate the view matrix, which translates your world coordinates into coordinates relative to the camera, by taking in consideration the location and rotation of the camera. It also calculates the projection matrix, which converts the world coordinates to the normalized render coordinates in the range [-1,-1] to [+1,+1]. In 2D games, you mostly dont need the distinction between these two matrices and only need the combined transformation matrix instead. You can pass this matrix to the shader, for example, by calling `spriteBatch.setProjectionMatrix(camera.combined);`



You can have multiple camera's or viewports and likewise, you can also have multiple world coordinate systems. A typical game has at least two of those, namely:

GUI/HUD coordinatesPermalink

These are buttons, labels and such which are stationary and always visible on the screen. Often they involve rendering text. For example in Super Mario the clock is always visible in the upper right corner of the screen and does not move when mario moves in the game world.

Most commonly scene2d is used for the HUD, which means that you'd use a Viewport to define the coordinate system. This coordinates system is typically in a range that is close to the device resolution, to give the best results when rendering the font. These coordinates are called banana units. This camera is practically never moved or rotated, it sits stationary at a location so that the world coordinate (0, 0) is located at the bottom left corner of the screen.

Game coordinatesPermalink

This is what suits best for your game and is used to implement game logic. It is good practice to keep the values around one for the best floating point precision. For example, your main character is 1.8 meter in height, the tree is 10 meter in height, etc. If you are making a galactic game where units and distance are very high, you might want to use e.g. kilometers instead.

The camera is used to look into your game world, just like it would when you use a video camera in the real world. The camera can be moved, rotated and scaled to display another portion of the world on the screen.

Poly Haven is a small company based in South Africa, working with artists around the world.

Our goal is to create a constantly growing community-funded resource of open content, for complete freedom and usability by professionals and hobbyists alike.

Previously we ran HDRI Haven, Texture Haven and 3D Model Haven as separate independent projects, but ultimately decided we could serve the community better by joining forces and creating a single new platform: Poly Haven.

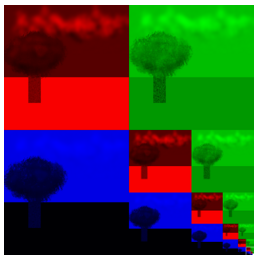
The origin of the term mipmap is an initialism of the Latin phrase *multum in parvo* ("much in little"), and map, modeled on bitmap.1(https://en.wikipedia.org/wiki/Mipmap#cite_note-staff-1) The term *pyramids* is still commonly used in a GIS context. In GIS software, pyramids are primarily used for speeding up rendering times.

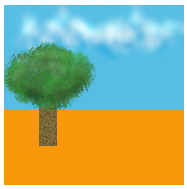
Mechanism

Example mipmap image storage: the principal image on the left is accompanied by filtered copies of reduced size.

Each bitmap image of the mipmap set is a downsized duplicate of the main texture, but at a certain reduced level of detail. Although the main texture would still be used when the view is sufficient to render it in full detail, the renderer will switch to a suitable mipmap image (or in fact, interpolate between the two nearest, if trilinear filtering is activated) when the texture is viewed from a distance or at a small size. Rendering speed increases since the number of texture pixels (*[texels](https://en.wikipedia.org/wiki/Texel*(graphics) "Texel (graphics)")_) being processed per display pixel can be much lower for similar results with the simpler mipmap textures. If using a limited number of texture samples per display pixel (as is the case with bilinear filtering) then artifacts are reduced since the mipmap images are effectively already anti-aliased. Scaling down and up is made more efficient with mipmaps as well.

If the texture has a basic size of 256 by 256 pixels, then the associated mipmap set may contain a series of 8 images, each one-fourth the total area of the previous one: 128×128 pixels, 64×64, 32×32, 16×16, 8×8, 4×4, 2×2, 1×1 (a single pixel). If, for example, a scene is rendering this texture in a space of 40×40 pixels, then either a scaled-up version of the 32×32 (without trilinear interpolation) or an interpolation of the 64×64 and the 32×32 mipmaps (with trilinear interpolation) would be used. The simplest way to generate these textures is by successive averaging; however, more sophisticated algorithms (perhaps based on signal processing and Fourier transforms) can also be used.





Showing each color channel of each level of an RGB mipmap as a separate plane (left) demonstrates that the whole mipmap forms a square of 4 times the area. As each plane requires 1⁄3 the storage, mipmaps therefore require 4⁄3 the memory; i.e., 1⁄3 ≈ 33% more.

The increase in storage space required for all of these mipmaps is a third of the original texture, because the sum of the areas 1/4 + 1/16 + 1/64 + 1/256 + ··· converges to 1/3. In the case of an RGB image with three channels stored as separate planes, the total mipmap can be visualized as fitting neatly into a square area twice as large as the dimensions of the original image on each side (twice as large on each side is four times the original area - one plane of the original size for each of red, green and blue makes three times the original area, and then since the smaller textures take 1/3 of the original, 1/3 of three is one, so they will take the same total space as just one of the original red, green, or blue planes). This is the inspiration for the tag *multum in parvo*.

Uses

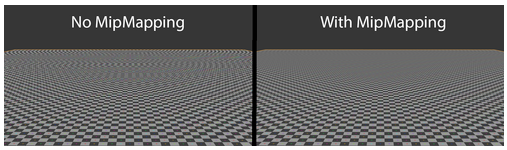


Image showing how mipmaps reduce aliasing at large distances (Aliasing causes a Moiré pattern in the left image.)

Mipmaps are used for:

- Level of detail (LOD)7(https://en.wikipedia.org/wiki/Mipmap#cite_note-7)[[8]](https://en.wikipedia.org/wiki/Mipmap#cite_note-8)
- Improving image quality. Rendering from large textures where only small, discontiguous subsets of texels are used can easily produce Moiré patterns;
- Speeding up rendering times, either by reducing the number of texels sampled to render each pixel, or increasing the memory locality of the samples taken;
- Reducing stress on the GPU or CPU.
- Water surface reflections9(https://en.wikipedia.org/wiki/Mipmap#cite_note-9)

Anisotropic filtering

Main article: Anisotropic filtering

When a texture is viewed at a steep angle, the filtering should not be uniform in each direction (it should be anisotropic rather than isotropic), and a compromise resolution is required. If a higher resolution is used, the cache coherence goes down, and the aliasing is increased in one direction, but the image tends to be clearer. If a lower resolution is used, the cache coherence is improved, but the image is overly blurry. This would be a tradeoff of MIP level of detail (LOD) for aliasing vs blurriness. However anisotropic filtering attempts to resolve this trade-off by sampling a non isotropic texture footprint for each pixel rather than merely adjusting the MIP LOD. This non isotropic texture sampling requires either a more sophisticated storage scheme or a summation of more texture fetches at higher frequencies.10(https://en.wikipedia.org/wiki/Mipmap#cite_note-10)